Throwing and Catching Exceptions

What happens when something goes wrong, and how a program responds

Every program we have written so far has assumed that things go right: the file exists, the number is a number, the salary is not negative. Real programs cannot assume that. These notes cover how Java reports a problem with an *exception*, the difference between checked and unchecked exceptions, and how to catch an exception, clean up with `finally`, and let try-with-resources close files for you. For reading, see Core Java, Volume I, Chapter 7.

A note before you begin. These are my notes from over the years, cleaned up so they are readable. They are not a study guide, and they are not gospel: not everything here appears in a lecture, and not every topic has notes yet. I write from memory and revise every semester. I also use AI to help convert documents to LaTeX, check for issues, and add explanations. I do not trust it completely, so I am still editing, and a typo or imprecise statement can survive longer than it should. Java also changes over time, though the basics we cover are resilient. If you find something that looks wrong, contradicts the book, or does not behave the way I claim when you run it, tell me. You will be doing the other students a favor, and you will understand the material better for having caught it. The textbook is the tiebreaker, and the compiler beats us both. These notes sometimes go beyond the book, but they do not replace the assigned chapters or the recorded lectures, and neither of those replaces the notes. Read the chapter, watch the lecture, and study the notes.

1. When Things Go Wrong

Problems in a running program come from a few places:

- **Bad input.** A user types `abc` where a number belongs, or a line in a file is missing a field.
- **The outside world.** A file does not exist, a disk is full, or a network connection drops.
- **Limits.** The program runs out of memory, or a recursive method calls itself until the stack overflows.
- **Bugs.** The code reads past the end of an array or calls a method on `null`.

An older way to report a problem is to return a special value, such as `-1` or `null`. That has two weaknesses. The caller can ignore the special value without any warning, and sometimes there is no special value to use, because every possible return value is a legitimate answer. Java's answer is the exception: an object that describes the problem and travels *separately* from the normal return value, so it cannot be mistaken for a result and cannot be silently lost.

Exceptions are for exceptional circumstances

Exceptions are not a tool for ordinary logic. Decisions belong in `if` statements, and a loop should end because its condition becomes false. An exception reports something that should not normally happen. Using one to steer a program is called using exceptions for *control flow*, and it is a mistake even when it works. This loop adds up an array of salaries:

```
double[] salaries = {60000, 90000, 72000};

// Do not do this: an exception is used to end the loop
double total = 0;
try {
    int i = 0;
    while (true) {
        total += salaries[i];
        i++;
    }
} catch (ArrayIndexOutOfBoundsException e) {
    // the loop "ends" here
}

// Do this: the loop condition ends the loop
double total2 = 0;
for (int i = 0; i < salaries.length; i++) {
    total2 += salaries[i];
}
```

Both versions print `222000.0`, but the first one is wrong for three reasons.

- **It hides what the code means.** A reader expects a loop to stop because of its condition. Here the real stopping rule is buried in a `catch` block.
- **It hides real bugs.** Suppose the loop body is mistyped as `salaries[i + 1]`. The second version would fail at once. The first version silently prints `162000.0`, because the same `catch` block swallows the genuine mistake.
- **It is slow.** Creating an exception records the whole call stack, which costs far more than checking a condition.

2. Throwing an Exception

You have already seen exceptions thrown on purpose. The `SalariedEmployee` constructor from the inheritance unit refuses a negative salary. Here is the same idea in a small method:

```
public class ThrowDemo {
    static double monthlyPay(double annualSalary) {
        if (annualSalary < 0) {
            throw new IllegalArgumentException("negative salary: " + annualSalary);
        }
        return annualSalary / 12;
    }

    public static void main(String[] args) {
        System.out.println(monthlyPay(60000));
        System.out.println(monthlyPay(-5000));
        System.out.println("done");
    }
}
```

```
5000.0
Exception in thread "main" java.lang.IllegalArgumentException: negative salary: -5000.0
        at ThrowDemo.monthlyPay(ThrowDemo.java:4)
        at ThrowDemo.main(ThrowDemo.java:11)
```

Notice what happened, and what did not:

- `throw` creates nothing special. `new IllegalArgumentException(...)` is an ordinary object creation, and `throw` hands that object to Java's exception mechanism.
- The method stopped at the `throw`. It never reached `return`.
- The exception traveled back up the chain of method calls, called the *call stack*, looking for code that would handle it. Nothing in `main` handled it, so the program stopped, and `"done"` was never printed.

Reading a stack trace

The output is a *stack trace*, and learning to read one saves hours. The first line names the exception's class and its message. Each `at` line is one method call that was in progress, most recent first, with the file and line number. Start at the top and look for the first line that names your own code. Here that is line 4 of `ThrowDemo.java`, inside `monthlyPay`, which was called from line 11 in `main`.

3. The Exception Hierarchy

Exceptions are objects, so they have classes, and those classes form an inheritance hierarchy. Everything that can be thrown descends from `Throwable`, which has two branches.

- `Error` is for serious problems inside the Java runtime, such as running out of memory. Your programs should not try to catch these.
- `Exception` is for everything else, and it has one special subclass, `RuntimeException`, for problems that usually come from a mistake in the code.

Table 1: Some exception classes you will meet, and which branch each belongs to.

Class	Superclass	Checked?	Typical cause
`OutOfMemoryError`	`Error`, indirectly	No	The program used all available memory
`StackOverflowError`	`Error`, indirectly	No	Recursion that never stops
`IOException`	`Exception`	Yes	Reading or writing failed
`FileNotFoundException`	`IOException`	Yes	A `PrintWriter` or `Scanner` could not open a file
`NoSuchFileException`	`IOException`, indirectly	Yes	A file passed to `Files.readString` does not exist
`IllegalArgumentException`	`RuntimeException`	No	A method received an argument it cannot accept
`NumberFormatException`	`IllegalArgumentException`	No	Parsing `"abc"` as a number
`NullPointerException`	`RuntimeException`	No	Calling a method on `null`
`ArrayIndexOutOfBoundsException`	`RuntimeException`, indirectly	No	An index past the end of an array
`ClassCastException`	`RuntimeException`	No	A bad downcast

Checked and unchecked

The most important line in that table is the "Checked?" column.

- **Unchecked exceptions** are `Error`, `RuntimeException`, and all of their subclasses. The compiler does not track them.
- **Checked exceptions** are every other subclass of `Exception`. The compiler *does* track them, and it enforces one rule: if a method can throw a checked exception, the method must either catch it or declare it.

A useful rule of thumb: a `RuntimeException` usually means the code is wrong. An array index out of bounds, a `null` you did not expect, and a bad cast can all be prevented by fixing the code. A checked exception usually means the world is wrong. No amount of careful coding guarantees that a file exists, so the compiler makes sure you planned for it.

4. Declaring Checked Exceptions

`Files.readString` can throw an `IOException`, which is checked. This method neither catches it nor declares it:

```
static String readStaff(String fileName) {        // does not compile
    return Files.readString(Path.of(fileName));
```

```
}
```

```
error: unreported exception IOException; must be caught or declared to be thrown
        return Files.readString(Path.of(fileName));
                                ^
```

One fix is to declare it with a `throws` clause. That passes the responsibility to whoever calls `readStaff`:

```
static String readStaff(String fileName) throws IOException {
    return Files.readString(Path.of(fileName));
}
```

You have written this clause before, on `main`, in the file I/O units. Declaring `throws IOException` on `main` works, but it only means that a missing file stops the program with a stack trace. The rest of these notes show how to do better.

A method declares only the checked exceptions it can throw and does not handle. It does not need to declare unchecked exceptions, although its Javadoc comment should still describe them with `@throws`, as the payroll classes did for a negative salary.

5. Catching Exceptions

To handle an exception, put the code that might throw it in a `try` block, and follow it with one or more `catch` blocks:

```
for (String name : new String[] {"files/staff.csv", "files/staf.csv"}) {
    try {
        String text = readStaff(name);
        System.out.println("Read " + text.lines().count() + " lines");
    } catch (NoSuchFileException e) {
        System.out.println("No such file: " + e.getMessage());
    } catch (IOException e) {
        System.out.println("Could not read the file: " + e.getMessage());
    }
}
```

```
Read 2 lines
No such file: files/staf.csv
```

If nothing in the `try` block throws, every `catch` block is skipped. If something throws, the rest of the `try` block is skipped, and Java runs the *first* `catch` block whose type matches. After that block finishes, the program continues after the whole statement. Here the loop moved on to the next file, and the program kept running. Every `catch` block is also a decision: should the program recover and continue, as this one does, or end cleanly? Part 2 covers how to make that decision.

Catch blocks are polymorphic

The second `catch` block is declared for `IOException`, but it would also catch a `FileNotFoundException`, an `AccessDeniedException`, or any other subclass. A `catch` parameter works just like a method parameter of a superclass type from the polymorphism notes: it accepts any object that is an `IOException`.

That is also why the order of `catch` blocks matters. If the general block came first, the specific block below it could never run, and the compiler rejects it:

```
} catch (IOException e) {
    System.out.println("Could not read the file");
} catch (NoSuchFileException e) {            // does not compile
    System.out.println("No such file");
}
```

```
error: exception NoSuchFileException has already been caught
```

Put the most specific types first and the most general last.

What to do with the exception object

The variable in a `catch` block, usually named `e`, is the exception object, and it has useful methods.

- `e.getMessage()` returns the message, such as the file name above.
- `e.getClass().getName()` returns the exception's class name.
- `e.printStackTrace()` prints the full stack trace. That is helpful while you are debugging, but a finished program should give its user a clear message instead, and record the details in a log, which Part 4 covers.

Catching several types in one block

When two exceptions need the same handling, one `catch` block can list both, separated by `|`:

```
static double parseSalary(String line) {
    String[] parts = line.split(",");
    return Double.parseDouble(parts[1]);
}
...
for (String line : new String[] {"Ada,60000", "Grace,abc", "Alan"}) {
    try {
        System.out.println(parseSalary(line));
    } catch (NumberFormatException | ArrayIndexOutOfBoundsException e) {
        System.out.println("Bad line \"" + line + "\": " + e.getClass().getSimpleName()
    );
    }
}
```

```
60000.0
```

```
Bad line "Grace,abc": NumberFormatException
Bad line "Alan": ArrayIndexOutOfBoundsException
```

The types in a multi-catch cannot be related by inheritance. `NoSuchFileException | IOException` is rejected, because the second already covers the first:

```
error: Alternatives in a multi-catch statement cannot be related by subclassing
```

6. The finally Clause

Sometimes code must run whether or not an exception happened: a file must be closed, or a connection released. A `finally` block runs after the `try` block and any `catch` block, every time.

```
static void process(String salary) {
    System.out.println("open the payroll file");
    try {
        System.out.println("monthly pay: " + Double.parseDouble(salary) / 12);
    } catch (NumberFormatException e) {
        System.out.println("not a number: " + salary);
    } finally {
        System.out.println("close the payroll file");
    }
}
...
process("60000");
System.out.println("---");
process("abc");
```

```
open the payroll file
monthly pay: 5000.0
close the payroll file
---
open the payroll file
not a number: abc
close the payroll file
```

The `finally` block runs even if the `try` block executes a `return`, and even if an exception is thrown that no `catch` block matches. A `try` block may have a `finally` block with no `catch` blocks at all.

Never return from a finally block. A `return` inside `finally` replaces whatever the `try` block returned, and it also throws away any exception that was on its way out. In this method, `parse("42")` and `parse("abc")` both return `-1`, and the `NumberFormatException` disappears without a trace: `try { return Integer.parseInt(s); } finally { return -1; }`. The compiler warns "finally clause cannot complete normally" when warnings are turned on, and Eclipse shows a similar warning.

7. try-with-resources

Closing files in `finally` works, but it is easy to get wrong. Since Java 7, the *try-with-resources* statement does it for you. Any object whose class implements the `AutoCloseable` interface can be declared in parentheses after `try`, and Java closes it when the block ends, whether it ends normally or with an exception. `PrintWriter`, `Scanner`, and the other file classes all implement `AutoCloseable`.

```
try (PrintWriter out = new PrintWriter("files/report.txt")) {
    out.println("Payroll Report");
    out.println("Ada 5000.00");
} catch (FileNotFoundException e) {
    System.out.println("Could not create the report: " + e.getMessage());
}
```

You have used this form since the file I/O units. Now you know what it does: it is a `try` statement with a built-in `finally` that calls `close()`. You can declare more than one resource, separated by semicolons, and they are closed in the reverse order from the order in which they were opened.

Suppressed exceptions

What if the block throws, and then `close()` throws too? Java keeps the first exception, the one that explains what went wrong, and attaches the second to it as a *suppressed* exception. This `Ledger` class is an `AutoCloseable` whose `close()` always fails:

```
static class Ledger implements AutoCloseable {
    Ledger() { System.out.println("ledger opened"); }

    void post(double amount) {
        if (amount < 0) {
            throw new IllegalArgumentException("negative amount: " + amount);
        }
        System.out.println("posted " + amount);
    }

    @Override
    public void close() {
        System.out.println("ledger closing");
        throw new IllegalStateException("ledger could not be saved");
    }
}
...
try (Ledger ledger = new Ledger()) {
    ledger.post(100);
    ledger.post(-5);
} catch (IllegalArgumentException e) {
    System.out.println("caught: " + e.getMessage());
    for (Throwable s : e.getSuppressed()) {
        System.out.println("suppressed: " + s.getMessage());
    }
}
```

```
ledger opened
posted 100.0
ledger closing
caught: negative amount: -5.0
suppressed: ledger could not be saved
```

Notice the order: the ledger closed *before* the `catch` block ran. A resource is always closed by the time any `catch` or `finally` block of the same statement runs.

8. Summary

Table 2: The exception keywords and what each does.

Keyword or form	What it does
`throw`	Sends an exception object up the call stack. The current method stops immediately.
`throws`	Declares, in a method header, the checked exceptions the method can pass to its caller.
`try` / `catch`	Runs code that might throw, and handles the exception types listed, most specific first.
`catch (A | B e)`	Handles two unrelated exception types with the same code.
`finally`	Runs after the `try` and `catch` blocks every time, for cleanup.
try-with-resources	Declares `AutoCloseable` resources that are closed automatically, in reverse order.

The one-sentence version: checked exceptions must be caught or declared, unchecked exceptions usually mean a bug to fix, and a `catch` block accepts any subclass of the type it names.

9. Try It

1. In `ThrowDemo`, wrap the second call to `monthlyPay` in a `try` block that catches `IllegalArgumentException` and prints its message. Does `"done"` print now? Why?
2. Change the `catch` in the multi-catch example to catch only `NumberFormatException`. What happens when the loop reaches `"Alan"`, and what does the stack trace tell you?
3. Rewrite the `finally` example so the file is a real `PrintWriter` opened with try-with-resources. Which lines of the original method are no longer needed?